

claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a75d4449d99d4e5a9.jsonl`  
- SHA-256: `972117b6fd6db890d77af8a3f0b270e62a03f1edb37ed39e52fc65fc2f7db502`  
- Source modified: `2026-07-02T03:04:59+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_41c8dc8a-8a4`  
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`
```

Transcript

1. user (2026-07-02T03:02:34.674Z)

You are an adversarial verifier in a tech-debt audit. Your default stance: REFUTE. Today is 2026-07-02.
A finder claims the following about the repo at C:/Users/avidu/Projects/khelsutra-guru/khelsutra (avidullu/khelsutra):

```
{  
  "title": "Issue #103 partially fixed and stale as written: name-on-reload + fps/resolution landed; size/duration remain",  
  "category": "hygiene",  
  "severity": "P2",  
  "file": "web/src/api/types.ts",  
  "line": 35,  
  "evidence": "Landed since filing: the reload-filename gap is fixed (persistent current-video label #127, App.tsx:159-172 resolves the name from GET /api/videos on a deep-link reload) and fps/resolution ship in the library (#140: types.ts:44-45, lib/video.ts:33-38 videoSpec, VideoLibrary.tsx:79). Still missing: VideoSummary (types.ts:35-47) has no size or duration fields (engine contract gap), the current-video label shows name only (App.tsx:295-307), and there is no video-info panel during processing.",  
  "proposed_fix": "Update the issue to strike the landed parts, then scope the remainder: engine exposes duration/size (metadata already probed at validation per the issue) on GET /api/videos or the job result; SPA renders a compact info panel from upload -> result. Cross-repo M.",  
  "effort": "M",  
  "tracked_in": "khelsutra#103"  
}
```

Repo root candidates: C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer, C:/Users/avidu/Projects/khelsutra-guru/khelsutra, C:/Users/avidu/Projects/khelsutra-guru/sports-data-collector, C:/Users/avidu/Projects/khelsutra-guru/rally-annotator, C:/Users/avidu/Projects/khelsutra-guru/sports-obsreport (pick from the finding's repo field; non-git dirs: 'C:/Users/avidu/Projects/khelsutra-guru/Annotation Setup', C:/Users/avidu/Projects/khelsutra-guru/khelsutra-screencast).

VERIFY against the CURRENT working tree (just pulled to origin master/main): (1) does the cited code actually exist and behave as claimed at file:line? (2) was it already fixed by a recent commit (git log --oneline -15 -- <file>)? (3) is the severity honest - construct the concrete failure scenario or downgrade; (4) is the proposed fix sane and correctly scoped; (5) is it owner-gated per: alpha/serving go-live, Studio P10/P11 corpus-promotion/training, product/strategy/licensing, real cloud spend, counsel ToS/DPA?

HARD RULES: READ-ONLY (no file modifications, no installs, no test-suite runs). gh read-only commands allowed. Your reasoning MUST cite file:line you actually read. CONFIRMED requires you personally saw the defect/debt in current code; if you cannot reproduce the claim from the code, say REFUTED; if a commit fixed it, ALREADY_FIXED.

2. user (2026-07-02T03:02:42.749Z)

```

1      // Types mirror the engine's verified contract (badminton-highlight-indexer).
2      // Anchors: backend/infrastructure/database.py:124-135 (play_segments),
3      // backend/main.py:331-396 (query/compile), :307-328 (highlights), :121 (config).
4
5      export interface RallyMetadata {
6          /** Gemini's shuttle_exchange_count (fallback max(3, dur/0.8)); absent on some
detectors. */
7          shot_count?: number;
8          /** Gemini confidence STRING (often "Unknown") ? not a numeric meter. */
9          shot_count_confidence?: string;
10         detector?: string;
11         [key: string]: unknown;
12     }
13
14     export interface RallySegment {
15         /** The integer the UI selects and passes to /api/compile as segment_ids. */
16         id: number;
17         start_time: number;
18         end_time: number;
19         duration: number;
20         /** motion-segmenter FABRICATION (random) ? DO NOT surface (see
PER_RALLY_SELECTION.md). */
21         server: string | null;
22         /** motion-segmenter FABRICATION (random) ? DO NOT surface. */
23         receiver: string | null;
24         metadata: RallyMetadata;
25         /** motion-only fabricated sub-events ? DO NOT surface. */
26         events?: unknown[];
27     }
28
29     export interface QueryResponse {
30         play_segments: RallySegment[];
31     }
32
33     /** One row of the indexed-video library. GET /api/videos [backend/main.py:447] returns
the most
34     * recent first so the console survives a reload. Every field but `video_id` is
optional/defensive. */
35     export interface VideoSummary {
36         /** The MD5 join key (the contract field). Optional defensively ? an older/regressed
engine may
37         * send only the raw `id`; read it through `lib/video.ts::videoId`, never directly.
*/
38         video_id?: string;
39         /** Raw DB fields an engine may send instead of the enriched contract (resilience
fallbacks). */
40         id?: string;
41         filepath?: string | null;
42         match_name?: string | null;
43         local_path?: string | null;
44         fps?: number | null;
45         resolution?: string | null;
46         created_at?: string | null;
47     }
48
49     export interface VideosResponse {
50         videos: VideoSummary[];
51     }
52
53     /** POST /api/annotations ? persist finished rally labels for a video as the annotator
CSV (P8).
54     * The engine writes the CSV verbatim (byte-compatible with the pipeline), keyed by the
file MD5. */
55     export interface AnnotationsResponse {
56         status: string;

```

```

57     num_rows?: number;
58     csv_uri?: string;
59     message?: string;
60 }
61
62 export interface CompileResponse {
63     status: string;
64     /** server-local path (NOT a URL) ? build the download URL with highlightUrl(). */
65     filepath: string;
66 }
67
68 /** The terminal RESULT of an async reel compile (#329): the job's `result` payload,
surfaced by
69 * GET /api/jobs/{id} once the stitch finishes. `status` is the pipeline verdict
("success" |
70 * "failed"); a "failed" result carries a `message`. The SPA builds the download URL
with
71 * highlightUrl() (split-origin-safe), but the engine also returns a same-origin
`download_url`. */
72 export interface CompileResult {
73     status: string;
74     filepath?: string;
75     download_url?: string;
76     message?: string;
77     video_id?: string;
78 }
79
80 export interface EngineConfig {
81     stitching?: {
82         /** The shot-count floor /api/compile silently enforces ? surface it, don't hide it.
*/
83         min_shot_count?: number;
84         [key: string]: unknown;
85     };
86     [key: string]: unknown;
87 }
88
89 /** GET /api/capabilities [backend/main.py] ? an HONEST probe of what THIS box can do,
rendered by
90 * the first-run wizard (PACKAGING_PLAN.md P2c). Presence-booleans only (never the Gemini
key value).
91 * Every field optional/defensive: a missing field degrades the wizard gracefully. */
92 export interface EngineCapabilities {
93     api_version?: number;
94     engine_version?: string;
95     segmenters?: string[];
96     default_segmenter?: string;
97     ffmpeg?: { ffmpeg?: boolean; ffprobe?: boolean; ffmpeg_path?: string; ffprobe_path?:
string };
98     gpu?: { nvidia_smi_present?: boolean };
99     /** Whether a Gemini key is PRESENT (never its value) ? drives the AI-highlights step.
*/
100     gemini_key_present?: boolean;
101     wasb_weights_present?: boolean;
102     /** `cloud_available` = can THIS server satisfy a per-request compute="cloud" dispatch
(a Cloud Run
103 * GPU Job)? The ingest compute-picker shows the "in the cloud" option only when
true. */
104     deployment?: { profile?: string; compute_target?: string; cloud_available?: boolean };
105     /** Which storage media THIS box can store INTO (P3). The medium picker renders only
`usable`
106 * backends; `free_gb` is `null` for a cloud bucket (no readable capacity). Absent on
older engines
107 * ? the SPA degrades to local-only (no picker). */
108     storage?: StorageCapabilities;

```

```

109     [key: string]: unknown;
110 }
111
112 /** One storable medium reported by GET /api/capabilities (P3). Presence/usable bools
only ? no
113 * secrets. `id`/`kind` are the backend name (local/gcs/s3/gdrive). */
114 export interface StorageBackendInfo {
115     id: string;
116     kind: string;
117     label: string;
118     usable: boolean;
119     /** Free space in whole GB, or `null` when unknowable (a cloud bucket). */
120     free_gb?: number | null;
121 }
122
123 export interface StorageCapabilities {
124     default_medium?: string;
125     backends?: StorageBackendInfo[];
126 }
127
128 /** POST /api/assets ? 201 (P4). The video was stored into the chosen medium and
registered as a
129 * library asset keyed by the whole-file MD5 (`video_id`). `stored_uri` is the durable
location to
130 * feed into POST /api/process (a bucket/Drive URI, or a local path for "this
computer"). */
131 export interface AssetResponse {
132     video_id: string;
133     /** The medium backend the asset lives in (local | s3 | gcs | gdrive). */
134     medium: string;
135     /** The durable, MD5-keyed location ? pass as video_path to processVideo(). */
136     stored_uri: string;
137     /** True when a durable copy was made (uploaded ? remote medium); false when
registered in place. */
138     copied: boolean;
139     sport?: string | null;
140     status?: string;
141 }
142
143 /** POST /api/process ? 202 [backend/main.py:260]. The job is queued; poll GET
/api/jobs/{job_id}. */
144 export interface ProcessResponse {
145     job_id: string;
146     /** Always "queued" at submit. */
147     status: string;
148     /** Relative poll path, e.g. "/api/jobs/<id>". */
149     poll: string;
150 }
151
152 /** Chunked upload (backend/api/uploads.py). The browser uploads a local file in
sequential parts
153 * ? max_part_mb, then `complete` returns a server-host `path` to feed straight into
POST /api/process. */
154 export interface UploadInitResponse {
155     upload_id: string;
156     /** Per-part ceiling in MB ? slice the file into parts this big. */
157     max_part_mb: number;
158     next_part: string;
159 }
160 export interface UploadPartResponse {
161     upload_id: string;
162     received_parts: number;
163     received_bytes: number;
164 }
165 export interface UploadCompleteResponse {

```

```

166     /** Absolute path on the engine host ? pass as video_path to POST /api/process. */
167     path: string;
168     video_id: string;
169     size_bytes: number;
170 }
171
172 /** GET /api/jobs/{job_id} [backend/main.py:303]. Two status layers:
173  * `status` = the EXECUTION state; `result.status` = the pipeline VERDICT
(success|failed).
174  * A `done` job whose `result.status` === "failed" (or with no video_id) is a success-
shaped
175  * FAILURE ? the UI must not treat it as a load. */
176 export interface JobStatus {
177     job_id: string;
178     status: "queued" | "running" | "done" | "failed";
179     /** Free-text coarse phase ("hashing", "segmenting (gemini)", "finished") ? NOT a
percentage. */
180     progress?: string | null;
181     /** The canonical video_id, surfaced top-level on success (also in result.video_id).
*/
182     video_id?: string | null;
183     /** Execution error (worker crash / unexpected) when status === "failed". */
184     error?: string | null;
185     result?: { status?: string; video_id?: string; message?: string } | null;
186 }
187

```

3. user (2026-07-02T03:02:44.822Z)

f9eea14 web: resilient video-summary readers + show fps in the library (#140)
9223ba9 docs: refresh khelsutra docs for today's storage-track work (P2-P5) (#139)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
a04a21b docs(tracker): inline the D14 decisions into the P2-P5 rows (#136)

4. user (2026-07-02T03:02:52.668Z)

```

140
141     // Stable chronological ordinal (by start_time) so re-sorting never rennumbers a rally.
142     const ordinalOf = useMemo(() => {
143         const m = new Map<number, number>();
144         [...rallies].sort((a, b) => a.start_time - b.start_time).forEach((r, i) =>
m.set(r.id, i + 1));
145         return (id: number) => m.get(id) ?? 0;
146     }, [rallies]);
147
148     // Load a video into the picker ? from a fresh ingest (IngestPanel) OR reopening a
stored one
149     // (VideoLibrary, which passes the match name). Rewrites ?video= so a refresh is
stable (D10).
150     const loadVideo = (id: string, label?: string) => {
151         setVideoId(id);
152         setVideoLabel(label ?? null);
153         window.history.replaceState(null, "", `?video=${encodeURIComponent(id)}`);
154     };
155
156     // Resolve the loaded video's name when we don't already have it (a deep-link ?video=
on reload, or
157     // an ingest hand-off) so the current-video label is always populated ? best-effort,
from the same
158     // GET /api/videos the library uses. A miss (not yet listed) keeps the friendly
fallback.
159     useEffect(() => {
160         if (!videoId || videoLabel) return;
161         let alive = true;

```

```

162     listVideos()
163     .then((vs) => {
164         const v = vs.find((x) => videoIdOf(x) === videoId);
165         const name = v ? videoName(v) : "";
166         if (alive && name) setVideoLabel(name);
167     })
168     .catch((err) => log("warn", "video.name_resolve_failed", { video_id: videoId,
error: String(err) }));
169     return () => {
170         alive = false;
171     };
172     }, [videoId, videoLabel]);
173
174     // Build ? POST /api/compile ? preview/download the stitched reel. One cid traces the
whole call.
175     const onBuild = async () => {
176         if (build.status === "building") return;
177         const cid = newCorrelationId();
178         const filename = reelFilename(reelName);
179         const ids = [...sel.selected];
180         if (!videoId) {
181             log("info", "build.no_video", { cid, selected: ids.length });
182             setBuild({
183                 status: "error",
184                 error: t("build.noVideo"),
185             });
186             return;
187         }
188         setBuild({ status: "building" });
189         log("info", "build.start", {
190             cid,
191             video_id: videoId,
192             filename,
193             selected: ids.length,
194             total_sec: Math.round(sel.totalSec),
195             below_floor: warning?.atRisk.length ?? 0,
196         });
197         try {
198             // Pass the active UI locale so the engine localizes the reel's branding watermark
(LW1).
199             const res = await compileReel(videoId, ids, filename, cid, i18n.resolvedLanguage
?? i18n.language);
200             const url = highlightUrl(videoId, filename);
201             setBuild({ status: "done", url, filename });
202             log("info", "build.done", { cid, video_id: videoId, filename, filepath:
res.filepath, url });
203         } catch (err) {
204             // Same friendly mapping the ingest flow uses: a bare 5xx / network throw reads
"is the engine
205             // running?", a real engine 4xx surfaces its own detail ? never a raw "HTTP 500"
(A5).
206             const status = err instanceof ApiError ? err.status : undefined;
207             const e = classifyError(err);
208             setBuild({ status: "error", error: errorText(t, e) });
209             log("error", "build.failed", { cid, video_id: videoId, filename, status, code:
e.code, detail: e.detail, error: String(err) });
210         }
211     };
212
213     // Every query application is ONE correlatable journey: a `query.parse` line records
exactly how
214     // the text was understood, then a single outcome line (apply / skip / empty) shares
the same cid.
215     // Unsupported concepts and zero-result queries are logged loudly ? never silently
dropped.

```

```

216     const onApplyQuery = (q: ParsedQuery, source: "typed" | "example") => {
217         const cid = newCorrelationId();
218         log("info", "query.parse", { cid, source, ...parseTrace(q) });
219
220         // Demand signal + honest "we ignored this": surface every shot-type/player/score
the user asked
221         // for but we can't yet honour (PER_RALLY_SELECTION.md §3 roadmap fields).
222         if (q.unavailable.length > 0) {
223             log("info", "query.unavailable_request", { cid, source, raw: q.raw, unavailable:
q.unavailable });
224         }
225
226         if (!q.recognized) {
227             log("info", "query.skip", { cid, source, raw: q.raw, reason: "no actionable
clause" });
228             return; // empty / unparseable ? leave the user's curation untouched
229         }
230
231         try {
232             const { ordered, selectedIds } = runQuery(rallies, q);
233             setOrder(ordered);
234             sel.apply(selectedIds);
235             log("info", "query.apply", {
236                 cid,
237                 source,
238                 filters: q.filters.length,
239                 sort: q.sort ? `${q.sort.key}:${q.sort.dir}` : null,
240                 limit: q.limit ?? null,
241                 total: rallies.length,
242                 selected: selectedIds.length,
243                 selected_ids: selectedIds,
244                 dropped_unavailable: q.unavailable.map((u) => `${u.kind}:${u.token}`),
245             });
246             if (selectedIds.length === 0) {
247                 // Loud: a recognized query that matches nothing makes the grid look empty ? say
why.
248                 log("warn", "query.empty_result", { cid, source, ...parseTrace(q) });
249             }
250         } catch (err) {
251             // Defensive: the pure parser/selector shouldn't throw, but never fail silently if
it does.
252             log("error", "query.apply_failed", { cid, source, raw: q.raw, error: String(err)
});
253         }
254     };
255
256     // Manual curation edits are traced too, so a journey (query ? hand-tune) reads as one
story.
257     const onToggle = (id: number) => {
258         log("debug", "rally.toggle", { id, action: sel.isSelected(id) ? "remove" : "add" });
259         sel.toggle(id);
260     };
261     const onBulk = (op: "select_all" | "clear" | "invert") => {
262         const resultCount = op === "select_all" ? rallies.length : op === "clear" ? 0 :
rallies.length - sel.count;
263         log("info", "selection.bulk", { cid: newCorrelationId(), op, result_count:
resultCount });
264         if (op === "select_all") sel.selectAll();
265         else if (op === "clear") sel.clearAll();
266         else sel.invert();
267     };
268
269     return (
270         <main className="min-h-screen bg-paper text-ink pb-24">
271         <header className="bg-ink text-white">

```

```

272         <div className="mx-auto flex max-w-5xl items-center justify-between gap-3 px-6
py-4">
273             <span className="text-xl font-extrabold tracking-tight">
274                 Khel<span className="text-green">?</span>Sutra <span className="font-
semibold text-lime">Studio</span>
275             </span>
276             <LanguageSwitcher />
277         </div>
278     </header>
279
280     <section className="mx-auto max-w-5xl px-6 py-10">
281         {/* Hard-blocker warning: FFmpeg absent ? indexing + reel compile can't run.
Persistent (unlike
282             the once-per-browser wizard), shown only when the engine positively reports
it missing. */}
283         <FfmpegBanner />
284         <span className="inline-flex rounded-full bg-green/15 px-3 py-1 text-xs font-
semibold text-green">
285             {t("app.badge")}
286         </span>
287         <h1 className="mt-4 text-3xl font-extrabold tracking-
tight">{t("app.title")}</h1>
288         <p className="mt-2 max-w-2xl text-ink/70">
289             <Trans i18nKey="app.intro" components={{ b: <strong />, i: <em /> }} />
290         </p>
291
292         {/* Persistent "current video" label ? always visible once a video is loaded, so
the operator
293             never loses track of which match they're editing (the library/ingest panel
that showed the
294             name has unmounted by now). Name from the library, else resolved from GET
/api/videos. */}
295         {videoId && (
296             <p
297                 className="mt-3 inline-flex items-center gap-2 rounded-full bg-ink/5 px-3
py-1 text-sm text-ink/70"
298                 data-testid="current-video"
299             >
300                 <span aria-hidden="true">?</span>
301                 <Trans
302                     i18nKey="app.currentVideo"
303                     values={{ name: videoLabel ?? t("app.currentVideoUnknown") }}
304                     components={{ b: <strong className="text-ink" /> }}
305                 />
306             </p>
307         )}
308
309         {/* P8 (flagged): switch the loaded video between picking rallies and annotating
them. */}
310         {videoId && annotateEnabled && (
311             <div className="mt-4 inline-flex rounded-full border border-black/10 p-1 text-
sm" data-testid="view-toggle">
312                 <button
313                     type="button"
314                     onClick={() => setView("pick")}
315                     aria-pressed={view === "pick"}
316                     className={"min-h-11 rounded-full px-4 " + (view === "pick" ? "bg-ink
text-white" : "text-ink/70")}
317                 >
318                     {t("annotate.tabPick")}
319                 </button>

```

5. user (2026-07-02T03:02:52.848Z)

```

1         // Resilient readers for a GET /api/videos row (VideoSummary). Defense-in-depth against

```



```

engine drift:
2      // the engine SHOULD return the contract fields (`video_id`, a friendly `match_name`,
`fps`,
3      // `resolution`), but if a build regresses to the raw DB shape (`id`, `filepath`, no
name), the library
4      // + the persistent current-video label still work ? they read through these helpers,
never the raw
5      // field. (The bug this guards against actually shipped once: the real engine returned
`id`/no-name
6      // while the e2e mock faked both, so a "green" UI was broken live.)
7      import type { VideoSummary } from "../api/types";
8
9      /** The MD5 join key ? falls back to the raw `id` column if `video_id` is absent. */
10     export function videoId(v: VideoSummary): string {
11         return v.video_id ?? v.id ?? "";
12     }
13
14     /** Filename stem of a path/URI: `~/My Match 2026.mp4` ? `My Match 2026`. Handles `/'
and `\",
15     * a trailing slash, and a no-extension name; returns "" for empty input. */
16     export function basenameStem(path: string | null | undefined): string {
17         const base =
18             String(path ?? "")
19                 .replace(/[/\\]+$/, "")
20                 .split(/[/\\]/)
21                 .pop() ?? "";
22         const dot = base.lastIndexOf(".");
23         return dot > 0 ? base.slice(0, dot) : base;
24     }
25
26     /** A human name ? the engine's `match_name`, else derived from the stored path/URI,
else "" (the
27     * caller applies its own localized "unnamed"/"your video" fallback). */
28     export function videoName(v: VideoSummary): string {
29         return v.match_name || basenameStem(v.local_path ?? v.filepath);
30     }
31
32     /** A compact spec line, e.g. `1920x1080 · 30fps` (either part optional; "" when neither
is known). */
33     export function videoSpec(v: VideoSummary): string {
34         const parts: string[] = [];
35         if (v.resolution) parts.push(v.resolution);
36         if (typeof v.fps === "number" && v.fps > 0) parts.push(`${Math.round(v.fps)}fps`);
37         return parts.join(" · ");
38     }
39

```

6. user (2026-07-02T03:03:01.652Z)

```

55     return (
56         <section className="mt-6" data-testid="video-library">
57             {Title}
58             <p className="mt-2 text-sm text-ink/50">{t("library.errorLoad")}</p>
59         </section>
60     );
61 }
62 if (state.videos.length === 0) {
63     return (
64         <section className="mt-6" data-testid="video-library">
65             {Title}
66             <p className="mt-2 text-sm text-ink/50">{t("library.empty")}</p>
67         </section>
68     );
69 }
70

```

```

71     return (
72       <section className="mt-6" data-testid="video-library">
73         {Title}
74         <p className="mt-1 text-sm text-ink/60">{t("library.intro")}</p>
75         <ul className="mt-3 grid grid-cols-[repeat(auto-fill,minmax(220px,1fr))] gap-3
list-none p-0 m-0">
76           {state.videos.map((v, i) => {
77             const id = videoId(v);
78             const name = videoName(v) || t("library.unsigned");
79             const meta = [videoSpec(v), v.created_at].filter(Boolean).join(" · ");
80             return (
81               <li key={id || `row-${i}`}>
82                 <button
83                   type="button"
84                   onClick={() => onSelect(id, name)}
85                   aria-label={t("library.openAria", { name })}
86                   data-testid="library-item"
87                   className="w-full text-left rounded-xl border border-black/10 bg-white
p-3 transition hover:border-green focus:outline-none focus:ring-2 focus:ring-green"
88                 >
89                   <div className="truncate font-semibold text-ink">{name}</div>
90                   {meta && <div className="mt-1 truncate text-xs text-
ink/50">{meta}</div>}
91                   <span className="mt-2 inline-flex items-center text-sm font-semibold
text-green">
92                     {t("library.open")} ?
93                   </span>
94                 </button>
95               </li>
96             );
97           })}
98         </ul>
99       </section>
100     );
101   }
102

```

7. user (2026-07-02T03:03:04.196Z)

{"body": "***Feedback from real use.** On reload the uploaded **filename isn't shown**, and it'd be good to show **stats about the video** ? name, size, fps, resolution, duration.\n\nExtends #99 (keep the filename visible during processing) with a richer **\"video info\" panel**.\n\n## Ground truth\nThe engine already probes the video during the validation phase (duration via `get\_video\_duration`; ffmpeg in the validators), so the metadata exists ? it just isn't surfaced. \n- **Engine:** expose video metadata (name, bytes, fps, WxH, duration, codec) via `/api/videos/{video\_id}` or on the job `result` (it's stored per `video\_id`).\n- **SPA:** render a compact video-info panel, shown from upload ? processing ? result, and **persisted across reload** (keyed by `video\_id`, like the rallies).\", \"comments\": [], \"labels\": [], \"state\": \"OPEN\", \"title\": \"SPA: show video stats (name, size, fps, resolution, duration) + persist them on reload\"}

8. user (2026-07-02T03:03:16.972Z)

f9eea14 web: resilient video-summary readers + show fps in the library (#140)
fb2f561 web: persistent "FFmpeg not detected" banner (hard-blocker warning) (#138)
cc06c32 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged) (#128)
9e9df72 feat(web): keep the video name visible after open (persistent current-video label) (#127)
39c1349 feat(web): P6 ? "Your videos" library (reopen a stored video) (#124)
7ae9cee feat(web): P2c first-run setup wizard (3 steps from /api/capabilities, 6 languages) (#88)
c1848df feat(web): LW2 ? send the UI locale to /api/compile (watermark) + /api/process (analytics) (#73)
46e0244 feat(web): A9 ? localized tap-only filter controls (#68)

f9eea14 web: resilient video-summary readers + show fps in the library (#140)
8667dbf P5: SPA storage-medium picker (store a video into your medium) (#137)
cc06c32 feat(web): P8 annotate-in-UI ? AnnotatePanel over the vendored core (flagged) (#128)
39c1349 feat(web): P6 ? "Your videos" library (reopen a stored video) (#124)
ed33a51 feat(build): poll the async /api/compile job for the reel (#329) (#108)
f67dfe7 feat(ingest): compute-picker ? choose local vs cloud GPU per run (item 3b) (#104)

9. user (2026-07-02T03:03:18.450Z)

```
khelsutra\web\src\App.tsx:40: { id: 1, start_time: 12, end_time: 20, duration: 8, server: null,
receiver: null, metadata: { shot_count: 9, shot_count_confidence: "High" } },
khelsutra\web\src\App.tsx:41: { id: 2, start_time: 41, end_time: 47, duration: 6, server: null,
receiver: null, metadata: { shot_count: 5 } },
khelsutra\web\src\App.tsx:42: { id: 3, start_time: 63, end_time: 84, duration: 21, server:
null, receiver: null, metadata: { shot_count: 19, shot_count_confidence: "Medium" } },
khelsutra\web\src\App.tsx:43: { id: 4, start_time: 95, end_time: 101, duration: 6, server:
null, receiver: null, metadata: {} },
khelsutra\web\src\App.tsx:44: { id: 5, start_time: 130, end_time: 142, duration: 12, server:
null, receiver: null, metadata: { shot_count: 11 } },
khelsutra\web\src\App.tsx:45: { id: 6, start_time: 158, end_time: 187, duration: 29, server:
null, receiver: null, metadata: { shot_count: 24, shot_count_confidence: "High" } },
khelsutra\web\src\components\A8.status.test.tsx:41: const share =
screen.getByTestId("whatsapp-share");
khelsutra\web\src\components\A8.status.test.tsx:51: const knShare =
screen.getByTestId("whatsapp-share");
khelsutra\web\src\api\client.test.ts:59: const seg = { id: 1, start_time: 0, end_time: 5,
duration: 5, server: null, receiver: null, metadata: { shot_count: 7 } };
khelsutra\web\src\components\AnnotatePanel.test.tsx:9: const v = screen.getByTestId("annotate-
video") as HTMLVideoElement;
khelsutra\web\src\components\AnnotatePanel.test.tsx:37:
fireEvent.click(screen.getByTestId("save-rally"));
khelsutra\web\src\components\AnnotatePanel.test.tsx:38: expect(screen.getByTestId("annotate-
op-error")).toBeTruthy();
khelsutra\web\src\components\AnnotatePanel.test.tsx:45:
fireEvent.click(screen.getByTestId("mark-start"));
khelsutra\web\src\components\AnnotatePanel.test.tsx:47:
fireEvent.click(screen.getByTestId("mark-end"));
khelsutra\web\src\components\AnnotatePanel.test.tsx:48:
fireEvent.click(screen.getByTestId("save-rally"));
khelsutra\web\src\components\AnnotatePanel.test.tsx:52: const recent =
screen.getByTestId("annotate-recent");
khelsutra\web\src\components\AnnotatePanel.test.tsx:61:
fireEvent.click(screen.getByTestId("mark-start"));
khelsutra\web\src\components\AnnotatePanel.test.tsx:63:
fireEvent.click(screen.getByTestId("mark-end"));
khelsutra\web\src\components\AnnotatePanel.test.tsx:64:
fireEvent.click(screen.getByTestId("save-rally"));
khelsutra\web\src\api\client.ts:247:/** Start a chunked upload. 400 = bad extension; 413 = over
the size limit [uploads.py:64]. */
khelsutra\web\src\api\client.ts:248:export async function uploadInit(filename: string,
totalSizeBytes: number, cid?: string): Promise<UploadInitResponse> {
khelsutra\web\src\api\client.ts:251: body: JSON.stringify({ filename, total_size_bytes:
totalSizeBytes }),
khelsutra\web\src\api\client.ts:256:/** Append ONE part (raw bytes ? not JSON). Parts are
strictly sequential 0,1,2,... [uploads.py:89]. */
khelsutra\web\src\api\client.ts:303: sentBytes: number;
khelsutra\web\src\api\client.ts:304: totalBytes: number;
khelsutra\web\src\api\client.ts:317: const init = await uploadInit(file.name, file.size, cid);
khelsutra\web\src\api\client.ts:321: const partBytes = partMb * 1024 * 1024;
khelsutra\web\src\api\client.ts:322: const totalParts = Math.max(1, Math.ceil(file.size /
partBytes));
khelsutra\web\src\api\client.ts:323: log("info", "upload.start", { cid, filename: file.name,
size: file.size, parts: totalParts, part_mb: init.max_part_mb });
khelsutra\web\src\api\client.ts:328: const chunk = file.slice(i * partBytes, Math.min((i +
1) * partBytes, file.size));
```

```

khelsutra\web\src\api\client.ts:330:      sent += chunk.size;
khelsutra\web\src\api\client.ts:331:      opts.onProgress?.({ sentBytes: sent, totalBytes:
file.size });
khelsutra\web\src\api\client.ts:334:      log("info", "upload.done", { cid, video_id:
done.video_id, path: done.path, size: done.size_bytes });
khelsutra\web\src\components\build-error.test.tsx:14:    { id: 1, start_time: 10, end_time: 20,
duration: 10, server: null, receiver: null, metadata: { shot_count: 12 } },
khelsutra\web\src\components\build-error.test.tsx:15:    { id: 2, start_time: 30, end_time: 45,
duration: 15, server: null, receiver: null, metadata: { shot_count: 14 } },
khelsutra\web\src\components\build-error.test.tsx:97:      const banner = await
screen.findByTestId("mode-banner");
khelsutra\web\src\components\FfmpegBanner.test.tsx:34:      await waitFor(() =>
expect(screen.getByTestId("ffmpeg-warning")).toBeTruthy());
khelsutra\web\src\components\FfmpegBanner.test.tsx:42:      await waitFor(() =>
expect(screen.getByTestId("ffmpeg-warning")).toBeTruthy());
khelsutra\web\src\components\FfmpegBanner.test.tsx:50:      expect(screen.queryByTestId("ffmpeg-
warning")).toBeNull();
khelsutra\web\src\components\FfmpegBanner.test.tsx:57:      expect(screen.queryByTestId("ffmpeg-
warning")).toBeNull();
khelsutra\web\src\components\FfmpegBanner.test.tsx:69:      expect(screen.queryByTestId("ffmpeg-
warning")).toBeNull();
khelsutra\web\src\api\types.ts:19:    duration: number;
khelsutra\web\src\api\types.ts:163:    received_bytes: number;
khelsutra\web\src\api\types.ts:169:    size_bytes: number;
khelsutra\web\src\api\upload.test.ts:2:// best-effort DELETE of the server-side partial on any
failure. Pins the part slicing (sizes +
khelsutra\web\src\api\upload.test.ts:19:/** A routing fetch over the /api/upload/* surface;
records the order + sizes of the calls. */
khelsutra\web\src\api\upload.test.ts:21:    const calls = { init: 0, parts: [] as number[],
partSizes: [] as number[], complete: null as number | null, deletes: 0 };
khelsutra\web\src\api\upload.test.ts:34:      const size = (init?.body as Blob).size;
khelsutra\web\src\api\upload.test.ts:36:      calls.partSizes.push(size);
khelsutra\web\src\api\upload.test.ts:37:      return ok({ upload_id: "up1", received_parts:
calls.parts.length, received_bytes: size });
khelsutra\web\src\api\upload.test.ts:41:      return ok({ path: "/srv/uploads/match.mp4",
video_id: "VID", size_bytes: 5 * MB });
khelsutra\web\src\api\upload.test.ts:52:const videoFile = (bytes: number, name = "match.mp4") =>
khelsutra\web\src\api\upload.test.ts:53:  new File([new Uint8Array(bytes)], name, { type:
"video/mp4" });
khelsutra\web\src\api\upload.test.ts:61:    const res = await uploadFile(videoFile(2.5 * MB), {
onProgress: (p) => progress.push(p.sentBytes) });
khelsutra\web\src\api\upload.test.ts:64:    expect(calls.partSizes).toEqual([1 * MB, 1 * MB, 0.5
* MB]); // last part is the remainder
khelsutra\web\src\components\IngestPanel.test.tsx:128:    expect(screen.getByTestId("ingest-
error")).toBeTruthy();
khelsutra\web\src\components\IngestPanel.test.tsx:141:    expect(screen.queryByTestId("compute-
picker")).toBeNull();
khelsutra\web\src\components\IngestPanel.test.tsx:155:    await screen.findByTestId("compute-
picker");
khelsutra\web\src\components\IngestPanel.test.tsx:191:      if
(/^\api\/upload\/up1\/part\/d+$/ .test(u)) return httpOk(200, { upload_id: "up1",
received_parts: 1, received_bytes: 3 });
khelsutra\web\src\components\IngestPanel.test.tsx:192:      if
(/^\api\/upload\/up1\/complete$/ .test(u)) return httpOk(200, { path: "/srv/up/match.mp4",
video_id: "mock", size_bytes: 3 });

```

[Showing results with pagination = limit: 60]

10. user (2026-07-02T03:03:31.145Z)

```

352:    duration_min: Optional[float] = None
488:    """Pull `` (fps, resolution)`` from the persisted validation results, or `` (None,
None)``. The
489:    ``resolution_fps_check`` validator already probed ffprobe and stored
``details={width,height,fps}``

```

```

492:         if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
494:             w, h, fps = d.get("width"), d.get("height"), d.get("fps")
495:             resolution = f"{w}x{h}" if w and h else None
496:             return (fps if isinstance(fps, (int, float)) else None), resolution
505:     ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
ffprobe
509:     fps, resolution = _video_dims(row.get("validation_results"))
516:     "fps": fps,
517:     "resolution": resolution,
521:@app.get("/api/videos")
522:def list_videos(request: Request, limit: Optional[int] = None):
561:     "size_bytes": st.st_size,
739:     ``GET /api/videos``.
788:     need = _remote_input_size_bytes(src_ref, getattr(cfg, "storage", None))
854:     # 3) Register the asset (keyed by MD5) so it appears in GET /api/videos ("my videos").
1334:def _remote_input_size_bytes(input_ref, storage_config) -> Optional[int]:
1364:     # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-
user,
1399:     need = _remote_input_size_bytes(
1446:@app.get("/api/videos/{video_id}/cost")
1528:     "duration_min": req.duration_min,
1548:     """"Shared compile resolution ? run at SUBMIT (fast-fail 4xx) AND in the WORKER (re-
resolve, since a
1739:     fps = cap.get(cv2.CAP_PROP_FPS)
1742:     start_frame = max(0, int((timestamp - 3.0) * fps))
1743:     end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))

```

11. user (2026-07-02T03:03:38.349Z)

```

485
486
487     def _video_dims(validation_results: Any) -> "tuple[Optional[float], Optional[str]]":
488         """"Pull ``(fps, resolution)`` from the persisted validation results, or ``(None,
None)``. The
489         ``resolution_fps_check`` validator already probed ffprobe and stored
``details={width,height,fps}``
490         on its result (every branch) ? so the videos row carries it with no schema change /
re-probe.""
491         for r in validation_results or []:
492             if isinstance(r, dict) and r.get("validator_name") == "resolution_fps_check":
493                 d = r.get("details") or {}
494                 w, h, fps = d.get("width"), d.get("height"), d.get("fps")
495                 resolution = f"{w}x{h}" if w and h else None
496                 return (fps if isinstance(fps, (int, float)) else None), resolution
497         return None, None
498
499
500     def _video_summary(row: Dict[str, Any]) -> Dict[str, Any]:
501         """"Map a DB video row to the API/SPA contract. ADDITIVE ? keeps the raw
``id``/``filepath`` (an
502         existing test + any old client rely on them) and adds the fields the SPA library +
the persistent
503         "current video" label actually read: ``video_id`` (the MD5 join key = ``id``), a
friendly
504         ``match_name`` (so an uploaded video shows its filename, not a generic fallback),
``local_path``,
505         ``created_at`` (from ``processed_at``), and ``fps``/``resolution`` (from the stored
ffprobe
506         validation details) so the library cards can show them. Mirrors the mock engine's
shape so mock and
507         real agree.""
508         fp = row.get("filepath") or ""
509         fps, resolution = _video_dims(row.get("validation_results"))
510         return {
511             **row,

```

```

512         "video_id": row.get("id"),
513         "match_name": _friendly_video_name(fp),
514         "local_path": fp,
515         "created_at": row.get("processed_at") or row.get("created_at"),
516         "fps": fps,
517         "resolution": resolution,
518     }
519
520
521     @app.get("/api/videos")
522     def list_videos(request: Request, limit: Optional[int] = None):
523         """List indexed videos (most recent first) so the console survives a reload ?
previously only a
524         single-row get existed (PACKAGING_PLAN.md P1). Rows are enriched to the SPA contract
525         (`video_id` + a friendly `match_name` derived from the filename) so the library
shows real
526         names and the persistent "current video" label survives an upload ? see
`video_summary`."""
527         return {"videos": [_video_summary(v) for v in
request.app.state.db.list_videos(limit=limit)]}
528
529
530     def _reels_dir(cfg) -> str:
531         return getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
532
533
534     _REEL_EXTS = (".mp4", ".mov")
535
536
537     @app.get("/api/reels")
538     def list_reels(request: Request):
539         """List compiled highlight reels present in the output dir, newest first, with a
download URL.

```

12. user (2026-07-02T03:03:48.020Z)

```

209:         duration REAL NOT NULL,
242:         duration REAL NOT NULL,
344:         # n_videos_at_fit ? held-out-user corpus size when fit (min_n trust floor
input)
562:         "INSERT INTO play_segments (video_id, start_time, end_time, duration,
server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
611:         (video_id, detector, chunk_index, start_time, end_time, duration,
673:         .execute("SELECT * FROM videos WHERE id = ?", (video_id,))
682:     def list_videos(self, limit: Optional[int] = None) -> List[Dict[str, Any]]:
687:         sql = "SELECT * FROM videos ORDER BY rowid DESC"
706:         - duration_min: float
714:         if filters.get("duration_min"):
715:             query += " AND r.duration >= ?"
716:             params.append(filters["duration_min"])

```

13. user (2026-07-02T03:03:54.516Z)

```

180         Checkpoints the WAL first so `dest` is self-contained. Returns `dest`."""
181         parent = os.path.dirname(os.path.abspath(dest))
182         if parent:
183             os.makedirs(parent, exist_ok=True)
184         with self._connect() as conn:
185             conn.execute("PRAGMA wal_checkpoint(FULL)")
186             shutil.copy2(self.db_path, dest)
187         return dest
188
189     def _create_base_tables(self, cursor):
190         """Create the videos / play_segments / events base tables (idempotent)."""
191         # Videos Table

```

```

192     cursor.execute("""
193         CREATE TABLE IF NOT EXISTS videos (
194             id TEXT PRIMARY KEY,
195             filepath TEXT NOT NULL,
196             processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
197             status TEXT NOT NULL,
198             validation_results TEXT
199         )
200     """)
201
202     # Play Segments Table (Extensible metadata JSON)
203     cursor.execute("""
204         CREATE TABLE IF NOT EXISTS play_segments (
205             id INTEGER PRIMARY KEY AUTOINCREMENT,
206             video_id TEXT NOT NULL,
207             start_time REAL NOT NULL,
208             end_time REAL NOT NULL,
209             duration REAL NOT NULL,
210             server TEXT,
211             receiver TEXT,
212             metadata TEXT,
213             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
214         )
215     """)
216
217     # Events Table
218     cursor.execute("""
219         CREATE TABLE IF NOT EXISTS events (
220             id INTEGER PRIMARY KEY AUTOINCREMENT,
221             segment_id INTEGER NOT NULL,
222             timestamp REAL NOT NULL,
223             type TEXT NOT NULL,
224             player TEXT,
225             details TEXT,
226             FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE CASCADE
227         )
228     """)
229
230     def _migrate_to_v1(self, cursor):
231         # v1: raw candidate detections (pre-confirmation), detector-agnostic.
232         # Common fields are columns; detector-specific metrics go in `stats` JSON,
233         # so a new segmenter reuses this table by setting its own `detector`/`stats`.
234         cursor.execute("""
235             CREATE TABLE IF NOT EXISTS candidate_segments (
236                 id INTEGER PRIMARY KEY AUTOINCREMENT,
237                 video_id TEXT NOT NULL,
238                 detector TEXT NOT NULL,
239                 chunk_index INTEGER,
240                 start_time REAL NOT NULL,
241                 end_time REAL NOT NULL,
242                 duration REAL NOT NULL,
243                 confidence REAL,
244                 stats TEXT,
245                 detection_params TEXT,
246                 confirmed_segment_id INTEGER,
247                 human_label TEXT,
248                 notes TEXT,
249                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

```

14. user (2026-07-02T03:04:19.623Z)

```

badminton-highlight-indexer\backend\main.py:492:         if isinstance(r, dict) and
r.get("validator_name") == "resolution_fps_check":
badminton-highlight-indexer\backend\main.py:1723:         print(f" {status_symbol}
{r.validator_name}: {r.message}")

```

```

badminton-highlight-indexer\backend\main.py:2085:         print(f"{status_symbol}
{r.validator_name}: {r.message}")
badminton-highlight-indexer\backend\worker\__main__.py:253:         + ";
".join(f"{r.validator_name}: {r.message}" for r in failures)
badminton-highlight-indexer\backend\worker\__main__.py:375:     probe with ffmpeg (same tool as
`get_video_duration`) and return None so the caller SKIPS
badminton-highlight-indexer\backend\reporting\harvest.py:92:         f"FAILED {getattr(r,
'validator_name', '?')}: {getattr(r, 'message', '')}"
badminton-highlight-indexer\backend\eval\audio\e1_probe.py:22: from backend.utils.video import
get_video_duration
badminton-highlight-indexer\backend\eval\audio\e1_probe.py:69:     dur =
get_video_duration(video) or (max((e for _, e in gts + wasb), default=0.0))
badminton-highlight-indexer\backend\eval\gemini_refine.py:35: from backend.utils.video import
extract_video_chunk, get_video_duration
badminton-highlight-indexer\backend\eval\gemini_refine.py:142:     duration =
get_video_duration(video)
badminton-highlight-indexer\backend\eval\gemini_refine.py:234:     duration =
get_video_duration(video)
badminton-highlight-indexer\backend\pipeline\validators\blur.py:23:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\blur.py:32:
validator_name=self.name, passed=False, message="Invalid frame count."
badminton-highlight-indexer\backend\pipeline\validators\blur.py:57:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\blur.py:76:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\blur.py:83:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\base.py:8:     validator_name: str
badminton-highlight-indexer\backend\pipeline\validators\base.py:69:
validator_name=val.name,
badminton-highlight-indexer\backend\pipeline\validators\scene_cut.py:23:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\scene_cut.py:35:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\scene_cut.py:97:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\scene_cut.py:104:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\pts_continuity.py:40:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\pts_continuity.py:60:
validator_name=self.name,
badminton-highlight-indexer\backend\pipeline\validators\pts_continuity.py:92:
validator_name=self.name,

```

[Showing results with pagination = limit: 25]

15. user (2026-07-02T03:04:58.908Z)

Structured output provided successfully